.Cross-Modal Generative Graph Self-Supervised Learning with Subgraph Sampling for Radar Jamming Recognition

Junjie Ding, Xiaobin Liu*, Qihua Wu, Caoyi Jiao, Xiaoxia Xie

Abstract:

Existing deep learning approaches for radar jamming recognition typically require abundant labeled data, making them costly to train. While self-supervised learning methods can utilize unlabeled data, standard contrastive approaches often require careful negative-sample construction and massive batch sizes. These requirements introduce severe instability in environments with high noise and limited signal diversity. A cross-modal, subgraph-sampled generative graph self-supervised learning framework is proposed. A cross-modal fusion encoder extracts and fuses features from pulse-compressed tensor and time-frequency image into a unified embedding. These embeddings are organized into a K-nearest-neighbor similarity graph. During pretraining, node attributes and graph connectivity are randomly masked, and a variational generative graph model is trained to reconstruct both features and structure. This generative objective avoids the instability of negative sampling and effectively captures higher-order relationships among signals. To address out-of-memory issues during full-graph pretraining, subgraph sampling is incorporated, enabling scalable mini-batch training while maintaining local structural consistency. The pretrained encoder is subsequently finetuned on a small labeled subset for jamming-type classification. Evaluations on simulated datasets across nine active jamming categories show that the proposed framework consistently outperforms standard supervised and self-supervised baselines, particularly under low jamming-to-noise ratios and few-label conditions.

Keywords: Radar jamming recognition; generative graph self-supervised learning; cross-modal fusion; masked graph modeling; subgraph sampling.

I. Introduction

Cognitive radar systems operate on a continuous perception-decision-action loop, constantly analyzing the electromagnetic environment to adapt their sensing strategies [1], [2]. Within this loop, automatic radar jamming recognition is a prerequisite for threat assessment and timely countermeasure selection, since active jammers can deliberately generate deceptive echoes or masking interference that undermines reliable detection and tracking. Recognizing these jamming types is inherently difficult due to diverse underlying mechanisms, extremely low jamming-to-noise ratios, and the high cost of obtaining reliable labels across different operational conditions. Consequently, the primary challenge is learning robust feature representations that maintain discriminative power even with limited annotations and severe noise.

Early radar jamming recognition methods treated feature engineering and classification as separate stages. Researchers typically constructed time–frequency (TF) or decomposition-based features by hand and then coupled them with shallow models for recognition [3], [4]. Generic machine-learning classifiers, including decision tree (DT) [5], support vector machine (SVM) [6], and random forest (RF) [7], provide efficient baselines for radar signal recognition, However, their performance hinges on the stability of handcrafted descriptors and can degrade under parameter drift, noise contamination, and waveform variations. This vulnerability motivated a shift toward data-driven, end-to-end representation learning.

Supervised deep learning has become a dominant route for radar jamming recognition, learning task-specific features from TF features or other radar-derived representations and reducing reliance on manually tuned descriptors [8], [9]. Recent supervised models strengthen representation learning from multiple directions. Lang et al. [10] fused global representation and local information in TF data via multi-head self-attention mechanism in the transformer. Lv et al. [11] combined a weighted ensemble CNN with transfer learning to improve robustness and generalization in deception jamming recognition. Li et al. [12] proposed a radar compound jamming recognition method based on image segmentation and a fused attention residual network, enabling finer localization and identification of overlapping interference patterns in composite jamming scenes. Simultaneously, recent fusion-oriented designs in cognitive radar increasingly exploit complementary domains through staged feature fusion [13] and decision fusion [14]. Such fusion can substantially enhance recognition robustness under low-JNR conditions by capturing the complementarity and redundancy of multiple modalities while accounting for their different predictive power and noise topology [15]. Despite these architectural improvements, supervised approaches fundamentally depend on large, reliably annotated datasets, severely restricting their deployment in label-scarce environments.

To mitigate the dependence on abundant, high-quality jamming labels, recent studies have begun to explore few-shot learning (FSL), which uses prior knowledge to generalize to new tasks from only a few supervised samples [16]. Du et al. [17] formulated SAR jamming recognition as a few-shot task in a metric-learning framework and proposed an aggregated-attention deformable convolutional network to capture long-range spatial contextual information and refine the representations of obscured features in the TF images. Luo et al. [18] designed a few-shot adaptive confidence aggregation and cross-modal refinement jamming recognition (JR-ACAR) method to mine complete information from jamming signal. Ding et al. [19] used conditional GAN augmentation to relieve data scarcity for jamming classification. Nevertheless, FSL still relies on labeled support sets; in contrast, self-supervised learning (SSL) can

learn transferable representations by exploiting information in large collections of unlabeled signals [20]. Most SSL methods are contrastive and can learn strong representations in practice [21], [22]. While contrastive methods have achieved strong empirical results, they rely heavily on informative negative samples and massive batch sizes. In high-noise environments with limited signal diversity, contrastive training becomes unstable. This limitation points to the need for relation-aware learning that explicitly models the relationships between different samples using graph neural networks [23].

Based on the above analysis, this paper proposes a cross-modal, subgraph-sampled, generative graph self-supervised learning framework for radar jamming recognition. To address information loss and ambiguity under low-JNR conditions, a cross-modal fusion encoder extracts features from the pulse-compression (PC) tensor and TF image and fuses them into a unified embedding. To convert unlabeled data into usable supervision, the embeddings are organized as a K-nearest-neighbor (KNN) graph, which explicitly exposes inter-sample similarity as a structural inductive bias. To avoid the instability of contrastive negatives, a masked generative pretraining task reconstructs randomly masked node attributes and graph topology [24], [25], encouraging the embedding to capture both semantic content and relational structure. Finally, subgraph sampling enables scalable mini-batch training on large similarity graphs while preserving local structural consistency. Under this framework, a large amount of unlabeled signal data is used for pretraining, and only a small labeled subset is required for effective fine-tuning.

The core contribution of this work is a unified framework that overcomes the limitations of single-modality inputs and label-dependent training in radar signal processing. A cross-modal fusion encoder is introduced that directly integrates pulse-compression tensors and time-frequency images, exploiting the distinct advantages of both representations. Furthermore, a subgraph-sampled masked generative graph self-supervised objective is developed that reconstructs node attributes and graph topology simultaneously. This approach effectively learns from unlabeled data without requiring negative-sample construction, yielding high training stability even in noisy environments. Extensive evaluations under low-JNR and few-label conditions demonstrate that the proposed approach consistently outperforms existing supervised and self-supervised baselines, providing a highly robust solution for real-world cognitive radar applications.

The remaining part of this article is organized as follows. Section II presents the radar jamming signal model and the corresponding data-processing method. Section III describes the proposed method in detail. Section IV reports the experimental setup and provides quantitative results and analyses. Section V concludes the paper.

II. Radar Jamming Model and Processing

This section presents the mathematical models of the radar jamming signals and introduces two data-processing methods.

A. Radar Jamming Model

This subsection summarizes common active jamming types based on the generation mechanisms of jamming signals and presents their mathematical models. The Linear Frequency Modulation (LFM) signal is used as the base waveform and generates diverse jamming signals via distinct generation functions, and its time-domain signal expression is as follows:

$$s(t) = \mathrm{rect}\left(\frac{t}{T_p}\right)\exp\left(j2\pi\left(f_0 t + \frac{1}{2}kt^2\right)\right)$$

where $\mathrm{rect}(\cdot)$ denotes the rectangular function, T_p is the pulse width, f_0 is the carrier frequency, and $k = \mathrm{B}/\mathrm{T}_p$ is the chirp rate. The received signal $x(t)$ can then be expressed as follows:

$$x(t) = j(t) + n(t)$$

where $j(t)$ denotes the jamming signal and $n(t)$ is circularly symmetric complex additive white Gaussian noise (AWGN).

1) Deception Jamming

Deceptive Jamming generates false targets by modulating the intercepted radar signal $s(t)$. Based on the digital radio frequency memory (DRFM) mechanism, the following types are considered:

Dense False Target Jamming (DFTJ) generates a cluster of false targets by superimposing multiple delayed replicas of the intercepted base signal to simulate a realistic multi-target environment. The jamming signal can be expressed as:

$$j_{DFTJ}(t) = \sum_{k=1}^{M} A_k \cdot s(t - \tau_k)$$

where A_k and τ_k represent the amplitude and the random time delay of the k-th false target, respectively, M is the number of false targets.

Interrupted sampling repeater jamming (ISRJ) generates coherent false targets by periodically sampling the radar signal and retransmitting it multiple times within each period. The jamming signal can be expressed as:

$$j_{ISRJ}(t) = \sum_{m=1}^{M}\sum_{n=0}^{N-1} \mathrm{rect}\left(\frac{t - (nM + n + m)T_i}{T_i}\right) s(t - mT_i)$$

where M denotes the number of retransmissions, N represents the total number of sampling slices, and T_i is the slice duration.

Smeared Spectrum Jamming (SMSPJ) is composed of n time-compressed sub-pulses. The duration of each sub-pulse is reduced to $1/n$ of the original pulse width T_p, while the frequency modulation slope is increased by a factor of n. This ensures each sub-pulse covers the full bandwidth. The jamming signal can be expressed as:

$$j_{SMSPJ}(t) = \sum_{i=1}^{n-1} p_i\left(t - i\frac{T_p}{n}\right)$$

$$j_i(t) = \exp\left[j\pi\left(n\frac{B}{T_p}t^2\right)\right],\ 0 \le t \le \frac{T_p}{n}$$

Where n represents the number of sub-pulses and B denotes the bandwidth.

Chopping and Interleaving Jamming (C&IJ) operates through two sequential stages: chopping and interleaving. In the chopping phase, the radar signal is segmented into equal-length slices via periodic sampling. In the interleaving phase, each slice is expanded into multiple sub-pulses, where the first sub-pulse replicates the sampled signal segment, and the subsequent sub-pulses are duplicates of the first one. The jamming signal can be expressed as:

$$j_{C\&IJ}(t) = \sum_{k=1}^{n} p\left(t - \frac{kT_p}{mn}\right)$$

$$p(t) = s(t)\left[\mathrm{rect}\left(\frac{t-\tau_a}{\tau_a}\right) * \sum_{i=0}^{m-1} \delta(t - iT_s)\right]$$

where m and n denote the number of slices and sub-pulses, Ts and τa denote the sampling period and sampling time of the sub-pulse, and $\delta(\cdot)$ is the impulse function.

2) Suppression Jamming

Suppression jamming can be categorized into coherent suppression jamming and non-coherent suppression jamming. Coherent suppression jamming modulates the retransmitted radar signal with noise, whereas non-coherent suppression jamming directly transmits high-power noise signals. This paper focuses on the coherent types, including Comb Spectrum Jamming (CSJ) and Smart Noise Jamming (SNJ).

CSJ is a typical suppression jamming technique designed to create a comb-like spectral distribution that effectively masks the target echo across the radar bandwidth. It is generated by modulating the intercepted LFM base signal with a multi-tone sinusoidal waveform. The jamming signal is expressed as:

$$j_{CSJ}(t) = U_{comb} \cdot \mathrm{rect}\left(\frac{t}{T_{comb}}\right) \sum_{k=1}^{m} a_k s(t) \cos(2\pi f_k t)$$

where U_{comb} and T_{comb} represent the jamming amplitude and pulse width, respectively; m denotes the number of frequency points; and f_k and a_k denote the frequency and amplitude of the k-th frequency point, respectively.

SNJ works by multiplying or convolving a time-delayed radar signal with narrow-band complex Gaussian noise, including noise product jamming (NPJ) and noise convolution jamming (NCJ). The jamming signal is expressed as:

$$j_{NP}(t) = s(t-\tau) \cdot n_j(t)$$

$$j_{NC}(t) = s(t-\tau) * n_j(t)$$

where τ denotes the time delay, $*$ denotes the convolution operation.

Noise modulation jamming generates interference by using Gaussian white noise to modulate the radar signal. This paper considers noise amplitude modulation jamming (AMJ) and noise frequency modulation jamming (FMJ), which can be expressed as follows:

$$j_{AMJ}(t) = \big(U_0 + A_n(t)\big)cos(2\pi f_0 t + \varphi_0)$$

$$j_{FMJ}(t) = U_0 \cos\left(2\pi f_0 t + 2\pi K_{PM} \int_0^t u(\tau)d\tau + \varphi_0\right)$$

where $A_n \sim N(\mu, \sigma_n^2)$ is the amplitude noise, $\varphi_0 \in U[0,2\pi]$, K_{PM} is the phase modulation factor, $u(\cdot)$ is the modulated noise.

B. Radar Jamming Processing

1) Pulse Compression:

To maximize the JNR and enhance the discriminability of coherent jamming features, PC is performed via matched filtering. The operation is defined as the convolution of the received signal $x(t)$ with the time-reversed complex conjugate of the transmitted signal $s(t)$:

$$y(t) = x(t) * h(t) = \int_{-\infty}^{+\infty} x(\tau)s^*(\tau - t)d\tau$$

Through this processing, the time-domain signal is effectively transformed into the range domain. In this domain, the signal energy is concentrated into narrow peaks, where the peak position corresponds to the target's range, thereby significantly suppressing noise and highlighting target characteristics.

2) Time–Frequency Transformation

To capture the time-varying spectral characteristics of non-stationary jamming signals, the short-time Fourier transform (STFT) is applied. By applying a sliding window $w(t)$ to localize the spectrum in time, the transformation is expressed as:

$$S(t,f) = \int_{-\infty}^{+\infty} x(\tau)w(\tau - t)e^{-j2\pi f\tau}d$$

This transformation maps the 1D signal into a joint TF domain distribution. It reveals the instantaneous frequency evolution and modulation patterns of the jamming signals, providing discriminative features that are otherwise obscured in the time domain.

III. PROPOSED METHOD

A cross-modal, subgraph-sampled, generative graph self-supervised learning framework is proposed to address the limitations of single-domain representations and label scarcity in radar jamming recognition.

A. Cross-Modal Fusion Encoder

To fully capture the characteristics of radar jamming signals, a cross-modal fusion encoder is designed, consisting of a signal branch, an image branch, and a fusion head. The overall framework is shown in Fig. 1.

1) Signal Branch

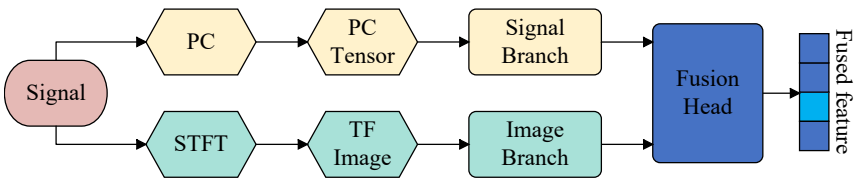

Fig. 1 Architecture of the Cross-Modal Fusion Encoder

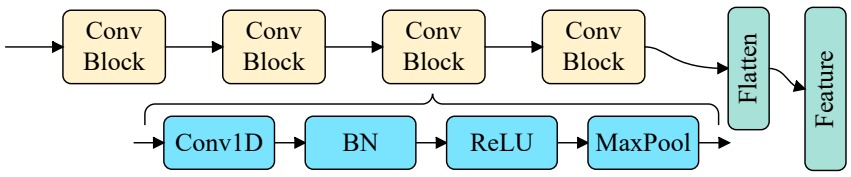

Fig. 2 Architecture of the Signal Branch

The raw input is a dual-channel tensor constructed from the real and imaginary components of the pulse-compressed signal, formulated as:

$$x_s = \left[\mathrm{R}\left(y(t)\right); \mathrm{I}\left(y(t)\right)\right] \in R^{2\times T}$$

where T denotes the fixed sequence length. The normalized tensor is input into a 1D Convolutional Neural Network. As shown in Fig. 2, this network is composed of four cascaded convolutional blocks, where each block sequentially performs 1D convolution, Batch Normalization (BN), ReLU activation, and Max Pooling. Finally, the obtained feature maps are flattened into a dense feature vector $z_s \in R^{d_s}$

2) Image Branch

The TF image generated by STFT is fed into a ResNet-18 network. ResNet-18 consists of an initial convolutional layer and four cascaded residual stages, where feature extraction is progressively performed through convolution, normalization, activation, and residual learning. Finally, the obtained feature maps are aggregated by global average pooling and mapped by linear projection into a dense feature vector $z_{tf} \in R^{d_{tf}}$.

3) Fusion Head

To effectively integrate the complementary information from both branches, the signal-brach feature z_s and the image-branch feature z_{tf} are concatenated to form a unified joint vector:

$$z_{in} = \mathrm{Concat}\left(z_s, z_{tf}\right)$$

This high-dimensional vector is then processed by two fully connected layers, where ReLU activation functions are configured to enhance non-linearity. This process fuses the cross-modal correlations and projects the features into a fused 512-dimensional vector, which serves both as the node descriptor for graph construction and as the input to the final classifier during fine-tuning.

B. Pretraining

In the pretraining stage, robust and structure-aware representations are learned from abundant unlabeled jamming signals by coupling the cross-modal fusion encoder with a masked generative graph modeling objective. As shown in Fig. 3, the fused embeddings are first organized into a global similarity graph that captures inter-sample relations in the representation space. To ensure scalability, optimization is performed on sampled subgraphs drawn from a stored neighborhood index rather than on the full adjacency matrix, enabling reconstruction-based self-supervision with bounded memory and computation while preserving local structural consistency.

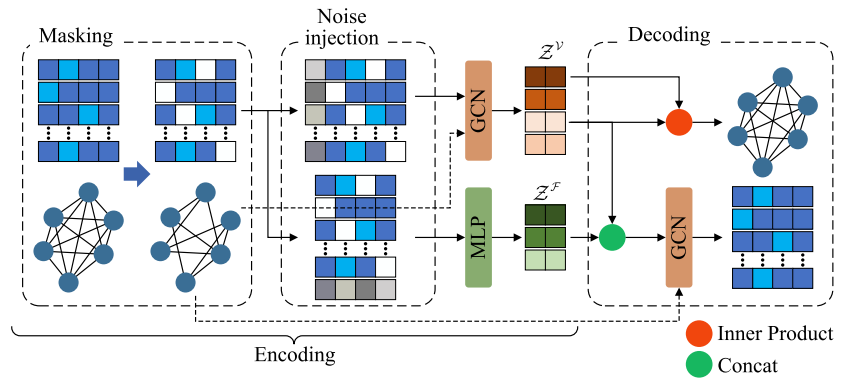

Fig. 3 Architecture of the Masked Variational Generative Graph Model

1) KNN Graph Topology Construction

Let $\{z_{fused}^{(i)}\}_{i=1}^{N}$ denote the fused embeddings produced by the cross-modal fusion encoder. Each embedding is projected by a learnable linear layer to obtain the node attribute $x_i \in \mathrm{R}^{\mathrm{d_x}}$, and stacking all attributes forms the node feature matrix :

$$X = [x_1; \ldots; x_N] \in R^{N\times d_x}$$

Pairwise semantic proximity is characterized by cosine similarity:

$$s_{ij} = \frac{x_i^T x_j}{||x_i||_2 ||x_j||_2}$$

For each node i, all candidates $j \neq i$ are ranked by s_{ij}, and the K most similar nodes are retained as its neighborhood:

$$\mathcal{N}_k(i) = TopK_{i\neq j}\left(s_{ij}\right)$$

To remain scalable for large N, similarity evaluation is performed in chunks and only the top-K neighbor indices per node are stored, forming a global neighborhood index $\mathrm{P} \in \mathrm{N}^{\mathrm{N\times K}}$. An undirected sparse KNN topology is induced by symmetrizing the directed neighbor relations, while removing self-connections:

$$A_{ij} = \begin{cases} 1, & j \in \mathcal{N}_k(i) \mathrm{or} i \in \mathcal{N}_k(j) \\ 0, & \text{otherwise} \end{cases}$$

2) Subgraph Sampling

To bound memory and computation, at each step, a seed set $\mathcal{S}$ is sampled and expanded using the stored KNN index to obtain a node set:

$$\mathcal{V}_s = \mathcal{S} \cup \bigcup_{i\in\mathcal{S}} \mathcal{N}_{k_s}(i), \ |\mathcal{V}_s| \leq N_{max}$$

The subgraph adjacency matrix A is assembled by retaining edges implied by **P** within $\mathcal{V}_s$ and enforcing symmetry. Node features X are obtained by re-encoding the samples indexed by $\mathcal{V}_s$ and applying the same projection, ensuring consistency with the current representation space, the global neighborhood index may also be refreshed at a fixed interval to mitigate representation drift.

3) Masked Variational Generative Graph Model

A generative graph self-supervised learning model is comparison to derive robust latent representations from the unlabeled

jamming signals. As illustrated in Fig. 3, this module follows an encode–decode paradigm.

The encoder starts by applying a random masking mechanism to the topological graph $G = (A, X)$, deliberately corrupting both the subgraph adjacency matrix A and the node features X with probabilities α_1 and α_2, respectively:

$$\tilde{A} = \mathrm{A} \odot M_{\alpha_1},\ \tilde{X} = \mathrm{X} \odot M_{\alpha_2}$$

Subsequently, Variational encoding is performed in two complementary latent spaces:

Node-level latent variables (Z^V): Noise ϵ_V is concatenated with the masked node features $\tilde{X}$, and $\left(\tilde{A}, [\tilde{X}; \epsilon_V]\right)$ is fed into a GCN encoder to produce per-node Gaussian parameters $(\mu^V, \log \sigma^{V2})$. Then Z^V is sampled using the reparameterization trick:

$$Z^V = \mu^V + \sigma^V \odot \eta,\ \eta \sim \mathcal{N}(0, I).$$

Global feature-level latent variables (Z^F): The masked attributes $\tilde{X}$are pooled to form a global attribute vector, which is concatenated with noise ϵ_F. An MLP predicts Gaussian parameters $(\mu^F, \log\ \sigma^{F2})$, and $z^F$is sampled using the reparameterization trick:

$$Z^F = \mu^F + \sigma^F \odot \xi,\ \xi \sim \mathcal{N}(0, I).$$

The decoder then aims to recover the original distribution through two parallel generative tasks, focusing on graph topology and semantic attributes, respectively:

Structure Reconstruction: The edge existence is assumed to follow a Bernoulli distribution. The probability of a link between node i and node j is recovered via the inner product of their node latent variables:

$$\mathrm{p}\left(A_{ij} \middle| Z^V\right) = \sigma\left((Z_i^V)^T Z_j^V\right)$$

where $\sigma(\cdot)$ denotes the sigmoid function.

Feature Reconstruction: To incorporate fine-grained semantic information, a fusion decoding scheme is utilized. Specifically, theglobal feature-level latent variable Z^F is broadcast and concatenated with the node-level latent variables Z^V to form a combined representation. This fused embedding is then processed by a GCN decoder to reconstruct the node feature matrix X:

$$\hat{X} = \mathrm{GCN}_{\mathrm{dec}}\left(A, \mathrm{Concat}(Z^V, Z^F_{\mathrm{broadcast}})\right)$$

To optimize the generative graph self-supervised learning module, the model minimizes a joint loss function $\mathcal{L}_{total}$:

$$\mathcal{L}_{total} = \lambda_A \mathcal{L}_{struct} + \lambda_X \mathcal{L}_{feat} + \lambda_{KL} \mathcal{L}_{\mathcal{KL}}$$

The definitions for each component are:

Structure Reconstruction Loss ($\mathcal{L}_{struct}$): This loss term supervises the decoder's ability to recover the original graph adjacency matrix A from the latent variables Z^V. Since the adjacency matrix of a graph is typically sparse, directly using standard cross-entropy loss would cause the model to be biased towards predicting "no edge." Therefore, positive sample weights are introduced into the binary cross-entropy (BCE) loss to balance these two classes of samples:

$$\mathcal{L}_{struct} = -\frac{1}{N^2} \sum_{i,j} \left[w_{pos} \cdot A_{ij} \log\left(p\left(A_{ij} \middle| Z^V\right)\right) + \left(1 - A_{ij}\right) \log\left(1 - p\left(A_{ij} \middle| Z^V\right)\right)\right]$$

where w_{pos} is the coefficient used to balance the weights of positive and negative samples.

Algorithm 1 Overall Pipeline of the Proposed Method

# Pretraining
for i = 1 to N do
→ Modality encoding: $h_{pc}^i = F_{\theta_{pc}}(u^i), h_{tf}^i = F_{\theta_{tf}}(u^i)$
→ Feature fusion: $z^i = F_{\theta_f}(h_p c^i, h_t f^i)$
→ Projection: $x^i = \Pi(z^i)$
→ Similarity: $s_{ij} = \kappa(x^i, x^j)$
→ Top-K: $P_i = TopK(s_{ij}, K)$
for training step t = 1, 2, … do
→ Subgraph sampling: $V = Sample(P,\ N_{max})$
→ Adjacency (sym.):

$$A_{ij} \coloneqq I\left((j \in P_i) \vee \left(i \in P_j\right)\right), \forall i, j \in V$$

→ Gather node features: $X = x^i | i \in V$
→ Masking: $\tilde{\mathrm{A}} = A \odot M_{\alpha 1}, \tilde{X} = X \odot M_{\alpha 2}$
→ Encoding:$(Z_v, Z_f) = Enc_\phi(\tilde{\mathrm{A}}, \tilde{X}, \eta, \xi)$
→ Decoding: $(\hat{\mathrm{A}}, \hat{X}) = Dec_\psi(A, Z_v, Z_f)$
→ Pre-training objective:

$$L_{total} = \lambda_A L_{struct} + \lambda_X L_{feat} + \lambda_{KL} L_{KL}$$

→ Update (θ, ϕ, ψ) :

$$(\theta, \varphi, \psi) \leftarrow \arg\min \mathcal{L}_{total}$$

→ if t mod R = 0, refresh P using the same steps above
→ Output: θ, φ, ψ
Fine-tuning
Input: $D_l = (u, y)$
→ Get fused feature $z = Enc_\theta(u)$
→ Update ω:

$$\omega \leftarrow arg\ min\ L_{cls}(F_\omega(z), y)$$

Output: ω
Inference
Input: query u_q
→$\hat{\mathrm{y}}_q = F_\omega\left(Enc_\theta(u_q)\right)$
Output: $\hat{\mathrm{y}}_q$

Feature Reconstruction Loss ($\mathcal{L}_{feat}$): This loss utilizes the Mean Squared Error (MSE) to measure the difference between the reconstructed features and the original features:

$$\mathcal{L}_{feat} = \frac{1}{N} \sum_{i=1}^{N} || x_i - \hat{x_i} ||_2^2$$

KL Divergence Loss ($\mathcal{L}_{\mathcal{KL}}$): This term measures the discrepancy between the latent distribution, denoted as $\mathcal{N}(\mu, \sigma^2)$, and the

standard normal distribution $\mathcal{N}(0, I)$, serving to regularize the latent space:

$$L_{KL} = -0.5 * \sum(1 + \log(\sigma^2) - \mu^2 - \sigma^2)$$

C. Fine-Tuning

A limited number of labeled signals are used in the fine-tuning stage, where the pretrained feature extraction module is directly connected to a two-layer MLP classifier, and a softmax function is applied to obtain the predicted class probabilities.

The framework is optimized by minimizing the Cross-Entropy Loss $\mathcal{L}_{cls}$ between the predicted probabilities and ground truth labels:

$$\mathcal{L}_{cls} = -\sum_{c=1}^{C} y_c \log(\hat{y_c})$$

Through this optimization process, the gradients back-propagated from $\mathcal{L}_{cls}$ dynamically refine the latent feature boundaries, enhancing the separability between different jamming categories.

D. Overall Pipeline

For unlabeled set and labeled set, the overall pipeline of the proposed method is presented in Algorithm 1. The learnable parameters are grouped as $\theta = \{\theta_{pc}, \theta_{tf}, \theta_f\}$, φ, ψ, and ω, where θ denotes the parameters of the fusion encoder φ and ψ denote the parameters of the graph encoder and graph decoder, and ω denotes the parameters of the classifier head.

TABLE I Detailed Jamming Generation Parameters

Type	Parameter	Value
DFTJ	Number of false targets	2-8
	False target delay	5μs
ISRJ	Number of sampling slices	2-6
SMSPJ	Number of sub-pulses	3-8
C&IJ	Number of pulses	2-4
	Number of time slots	2-5
CSJ	Number of comb lines	3-7
	Jamming amplitude factor	0.5-2.0
	Jamming pulse width factor	0.5-2.0
NPJ	Time delay	0-5μs
	Gaussian white noise	Mean:0 variance:1
NCJ	Time delay	0-5μs
	Gaussian white noise	Mean:0 variance:1
AMJ	Gaussian white noise	Mean:0 variance:1
FMJ	Gaussian white noise	Mean:0 variance:1

IV. SIMULATION RESULTS AND ANALYSIS

A. Experimental Setup

1) Dataset Settings

To evaluate the proposed method, a simulated radar jamming dataset is constructed based on the mathematical models in Section II. The dataset focuses on nine distinct categories of active jamming signals: DFTJ, ISRJ, SMSPJ, C&IJ, CSJ, NPJ, NCJ, AMJ, and FMJ. Fig. 4 shows the TF images of the LFM signal and randomly selected samples from the nine jamming types in the dataset.

The simulation settings are as follows: the sampling rate is fs=20MHz, the bandwidth is B=10MHz, and the pulse width is Tp=20μs. The JNR ranges from -20 dB to 0 dB in steps of 2 dB. For each category, 100 samples are generated at each JNR level for training and 30 samples are generated for testing. To prevent the model from overfitting to fixed patterns, the parameters of each jamming type are randomly sampled within predefined ranges for each instance. The detailed parameter configurations are listed in TABLE I.

2) Training Settings

Throughout training, a consistent optimization strategy is adopted to ensure stable convergence. Stochastic Gradient Descent (SGD) optimizer with a momentum of 0.9 and a weight decay of 1×10^{-4} is used to update the network parameters. The learning rate is initialized to 1.5×10^{-2} and scheduled by cosine annealing.

For graph construction, the neighborhood size is set to $K = 40$. For masked modeling, the masking coefficients are fixed as $\alpha_1 = \alpha_2 = 0.1$ in all experiments. For the training objective, the loss weights λ_A, λ_X and λ_{KL} are empirically set to 0.5, 1.0 and 0.05, respectively.

3) Baselines and Evaluation Metrics

The proposed method is compared with supervised CNN backbones and widely used self-supervised learning (SSL) methods under the same data split and training setting. For supervised baselines, ResNet, AlexNet, DenseNet, and VGG are trained end-to-end with cross-entropy. For SSL baselines, contrastive methods (SimCLR[21], MoCo v2[22]) and non-contrastive methods (BYOL[26], VICReg[27]) are included.

Overall accuracy (OA), precision, recall, F1-score, and kappa are reported. In addition, JNR-conditioned accuracy is reported by grouping the test samples according to JNR to evaluate robustness under different interference intensity levels.

4) Evaluation Protocols

To enable controlled analysis along the two key factors—label availability (ratio) and interference intensity (JNR)—the following evaluation protocols are adopted. The comparisons are conducted with a labeled ratio fixed at 0.01. For overall recognition performance (Section IV-B), overall accuracy is reported by aggregating all test samples across the full JNR range. For JNR-level evaluation (Section IV-C), the test set is grouped by JNR, and per-JNR accuracy is reported using the same trained model. For the few-label analysis (Section IV-D), the labeled ratio is varied over 0.01–0.1; at each ratio, labeled samples are selected in a class-balanced manner, and accuracy is computed on the same fixed test set aggregated

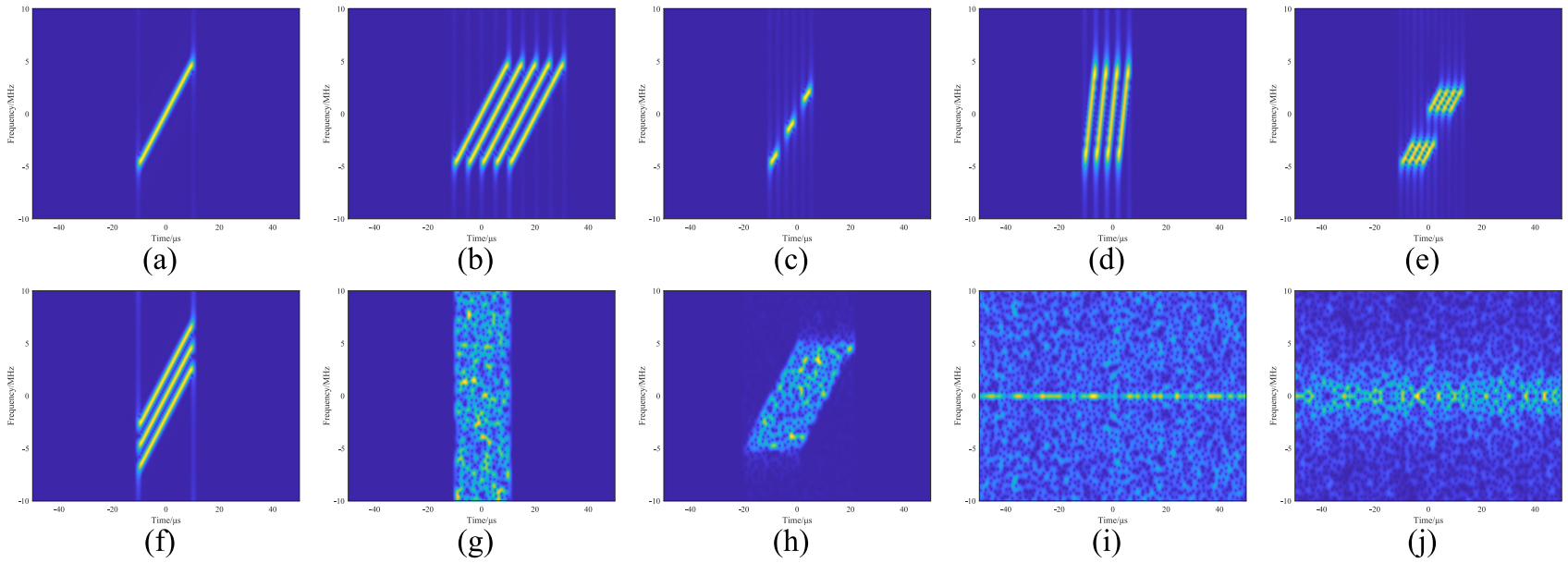

Fig. 4 TF Images of Signals. (a)LFM. (b) DFTJ. (c) ISRJ. (d) SMSPJ. (e) C&IJ. (f) CSJ. (g) NPJ. (h) NCJ. (i) AMJ. (j) FMJ.

over all JNR levels. Results are reported as the mean and variation over repeated runs with different random seeds.

B. Overall Recognition Performance

TABLE II summarizes the overall recognition performance of all compared methods. The proposed method achieves the best performance, with an OA of 75.13% and a kappa of 72.02%. Among the supervised baselines, DenseNet performs best, reaching an OA of 45.31% and a kappa of 38.47%. Among the SSL baselines, MoCo v2 achieves the best performance, with an OA of 58.42% and a kappa of 53.22%. Compared with these two baselines, the proposed method improves OA by 29.82 and 16.71 percentage points, respectively, and improves kappa by 33.55 and 18.80 points, respectively.

TABLE II OA, and kappa of Different Methods

Method	OA(%)	kappa(%)
ResNet	40.06±3.25	32.56±3.66
AlexNet	31.99±0.42	23.48±0.48
VGG	42.32±12.50	35.11±14.06
DenseNet	45.31±1.64	38.47±1.84
SimCLR	50.16±3.10	43.93±3.49
MoCo v2	58.42±0.93	53.22±1.05
BYOL	53.14±1.40	47.29±1.57
VICReg	53.33±0.87	47.50±0.98
Ours	75.13±1.59	72.02±1.79

The class-wise results in TABLE III-V further support this observation. In terms of macro-level performance, the proposed method achieves the highest precision (78.83%), recall (75.13%), and F1-score (75.52%) among all compared methods. Its advantage is particularly evident for challenging categories such as SMSPJ, DFTJ, CSJ, NCJ, and FMJ, where the F1-scores reach 75.14%, 75.80%, 69.97%, 69.52%, and 72.83%, respectively. The recall values for DFTJ (85.56%) and NCJ (81.01%) further indicate that the proposed method is effective in reducing missed detections in difficult classes. Overall, the improvement is observed across multiple classes rather than being concentrated in only a few categories, indicating the robustness of the learned representation.

C. Recognition Performance at Different JNR Levels

As shown in Fig. 5, the recognition accuracy of all methods generally improves as the JNR increases, indicating that the task becomes easier when the influence of noise is reduced. Among all compared methods, the proposed approach consistently achieves the best performance over the entire JNR range. Its advantage is particularly evident in the low- and medium-JNR regions, where discriminative features are more severely corrupted by noise and robust representation learning becomes crucial. In addition, the proposed method rises more rapidly and reaches a high-accuracy regime earlier than the other compared methods, demonstrating stronger robustness. Although the performance gaps gradually narrow at higher JNR levels as most methods approach saturation, the proposed method remains the best-performing approach overall.

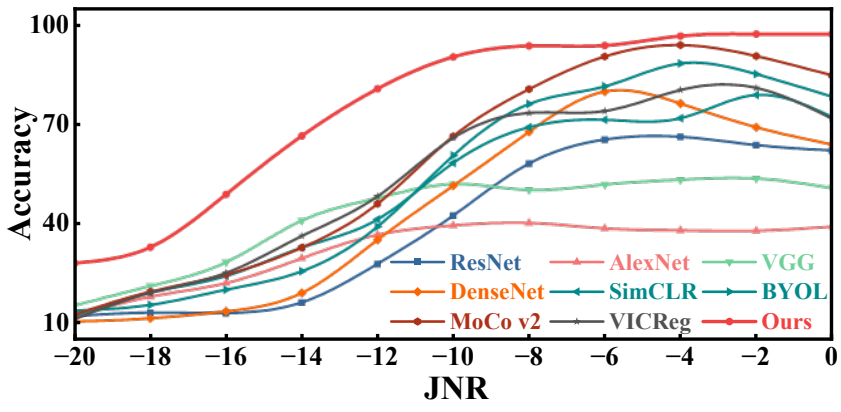

Fig. 5 Recognition Accuracy of Different Methods Under Varying JNR Levels

D. Few-Label Performance Analysis

TABLE VI reports the recognition accuracy under different labeled ratios. The proposed method achieves the highest accuracy across the entire range from 0.01 to 0.10, indicating that the proposed method remains effective under low-label conditions.. In the most challenging setting of 0.01, the proposed method reaches 75.13%, exceeding the best SSL baseline, MoCo v2 (58.42%), by 16.71 percentage points. This result suggests that the proposed pretraining strategy provides transferable representations even when only a very small amount of labeled data is available.

As the labeled ratio increases, the accuracy of all methods improves, while the proposed method maintains the best performance throughout. For example, at 0.05, the proposed method achieves 83.20%, exceeding the best competing baseline, VGG (75.26%), by 7.94 percentage points. At 0.10, it reaches 84.76%, which is still 8.04 percentage points higher than VGG (76.72%). These results

indicate that the advantage of the proposed framework is not limited to the most label-scarce setting, but is maintained as more labeled samples become available.

Another important observation is the training stability. Several baselines, especially the supervised CNN models, exhibit relatively large fluctuations under certain labeled ratios, suggesting sensitivity to sample selection and optimization when annotations are scarce. By contrast, the proposed method maintains relatively small performance variation across repeated trials, indicating more stable convergence in low-label settings.

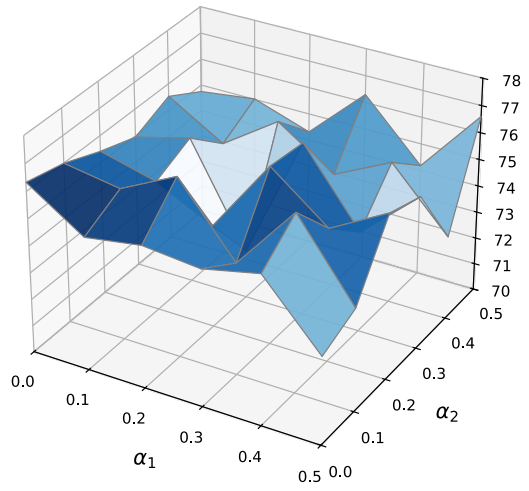

Fig. 6 Performance under different masking settings

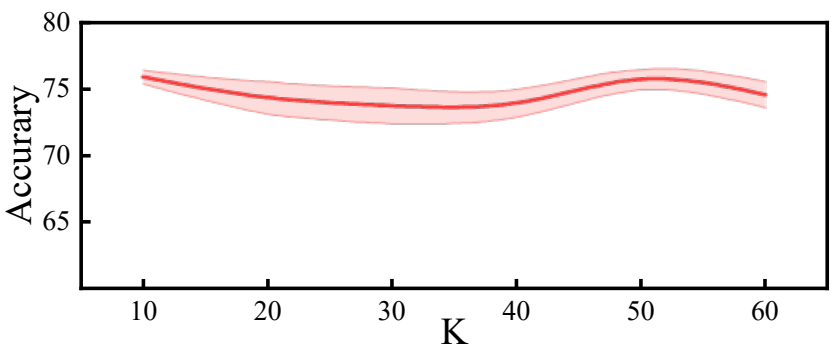

Fig. 7 Performance under different K

E. Ablation Studies

TABLE VII reports the ablation results of the proposed framework. The full model achieves the best performance, while all ablated variants show noticeable degradation. In particular, removing the pretraining stage reduces OA from 75.13% to 58.25%, indicating that the graph-based self-supervised pretraining plays a major role in learning effective representations.

The modality ablations further show that both branches contribute to the final performance. Removing the signal branch decreases OA to 44.67%, whereas removing the image branch decreases OA to 60.89%. These results suggest that the two branches provide complementary information, and that the gain of the proposed framework comes from the joint use of graph-based pretraining and cross-modal fusion rather than from a single component.

F. Hyperparameter Sensitivity Analysis

As shown in Fig. 6 and Fig. 7, the performance of the proposed framework remains relatively stable within the tested ranges of the masking coefficients and neighborhood size. Although minor fluctuations can be observed under some parameter settings, no severe performance degradation is observed overall. These results suggest that the proposed framework is not overly sensitive to these hyperparameters.

V. CONCLUSION

This paper proposes a cross-modal generative graph SSL framework with subgraph sampling for radar jamming recognition. A cross-modal fusion encoder combines PC tensors and TF images into compact embeddings, which are organized as a KNN similarity graph. The pretraining stage employs masked variational generative graph modeling to reconstruct both graph topology and node attributes, learning robust and structure-aware representations without negative sampling. Experiments on nine simulated jamming categories over JNRs from −20 to 0 dB demonstrate consistent improvements over supervised and self-supervised baselines, with particularly large gains under low JNR and few-label settings.

TABLE III Class-Wise Precision (%) of Different Methods

Type	Precision (%)								
	ResNet	AlexNet	VGG	DenseNet	SimCLR	MoCo v2	BYOL	VICReg	Ours
C&IJ	78.88±2.19	32.74±10.31	51.27±31.36	70.37±22.90	68.41±3.37	63.50±7.86	59.90±12.02	73.78±11.29	**84.15±6.30**
ISRJ	**99.69±0.22**	68.88±1.18	95.86±2.94	97.64±1.77	74.82±0.82	83.18±3.72	74.97±1.74	83.48±2.12	96.01±2.02
SMSPJ	22.79±2.71	23.18±1.29	38.83±4.77	74.38±18.61	38.60±1.22	63.54±1.23	55.15±6.38	46.45±2.22	**92.16±5.18**
DFTJ	41.92±11.72	31.03±43.89	33.17±7.23	66.59±3.37	33.53±9.83	49.91±14.19	40.01±13.36	39.07±10.90	**68.12±1.64**
CSJ	48.71±10.82	34.67±5.79	36.88±31.79	61.14±2.52	38.85±10.69	46.01±13.52	41.66±18.51	37.72±4.97	**79.75±23.00**
NPJ	16.89±1.75	0.00±0.00	46.38±4.80	14.53±0.39	50.56±4.86	63.90±11.79	63.97±23.14	58.19±8.93	**73.29±6.24**
NCJ	59.27±21.30	0.00±0.00	20.65±19.08	41.41±3.56	42.15±10.80	55.33±10.64	51.08±10.31	51.07±9.96	**61.46±7.80**
AMJ	95.75±3.18	24.91±1.26	45.50±16.41	**99.12±1.25**	64.91±8.75	70.15±1.17	74.94±14.22	66.57±8.42	80.87±10.78
FMJ	28.77±6.97	0.00±0.00	16.67±23.57	26.50±3.38	59.26±8.09	59.71±9.32	64.38±6.37	63.18±18.01	**73.67±8.55**
Average	54.74±6.76	23.94±7.08	42.80±15.77	61.30±6.42	52.34±6.49	61.69±8.16	58.45±11.78	57.72±8.54	**78.83±7.94**

TABLE IV Class-Wise Recall (%) of Different Methods

Type	Recall (%)								
	ResNet	AlexNet	VGG	DenseNet	SimCLR	MoCo v2	BYOL	VICReg	Ours
C&IJ	34.44±8.47	46.87±8.89	73.64±4.70	55.05±4.51	58.38±1.41	64.55±2.27	61.11±5.81	58.99±9.89	**79.80±6.13**
ISRJ	64.34±0.71	75.45±2.58	58.79±17.00	63.23±1.89	74.44±10.73	84.04±1.36	**85.25±2.30**	67.37±8.08	72.42±2.85

SMSPJ	58.89±2.93	19.60±8.75	38.99±27.41	58.18±3.22	35.35±15.42	45.56±10.57	38.99±7.93	38.38±14.09	**63.64±2.11**
DFTJ	31.11±6.36	2.73±3.86	39.19±21.13	39.39±1.29	37.68±16.26	48.48±10.73	38.69±14.12	43.74±14.42	**85.56±3.98**
CSJ	39.60±1.43	44.04±2.76	34.24±24.35	41.21±0.65	41.82±4.72	55.66±2.49	49.49±2.35	50.30±3.89	**66.16±8.78**
NPJ	26.36±11.32	0.00±0.00	22.63±21.56	44.24±4.54	47.37±8.42	50.40±8.48	43.43±11.23	53.64±3.98	**67.58±5.57**
NCJ	22.73±2.81	0.00±0.00	11.21±9.28	39.80±3.96	46.57±6.03	51.62±7.00	47.68±5.83	51.01±6.52	**81.01±1.65**
AMJ	52.83±0.52	**99.19±0.71**	81.21±23.14	44.44±1.36	56.97±6.90	64.95±1.22	60.10±3.13	61.41±13.18	86.57±7.65
FMJ	30.20±6.84	0.00±0.00	21.01±29.71	22.22±3.15	52.83±15.47	60.51±5.23	53.54±2.44	55.15±14.50	**73.43±6.56**
Average	40.06±4.60	31.99±3.06	42.32±19.81	45.31±2.73	50.16±9.48	58.42±5.48	53.14±6.13	53.33±9.84	**75.13±5.03**

TABLE V Class-Wise F1-score (%) of Different Methods

Type	F1-score (%)								
	ResNet	AlexNet	VGG	DenseNet	SimCLR	MoCo v2	BYOL	VICReg	Ours
C&IJ	47.46±8.56	36.16±4.75	54.24±19.45	59.09±7.84	62.99±2.24	63.72±3.66	59.55±6.39	65.30±9.53	**81.45±0.63**
ISRJ	78.21±0.57	71.99±1.18	71.30±12.01	76.75±1.94	74.23±5.75	**83.53±1.22**	79.73±0.34	74.19±3.85	82.53±2.14
SMSPJ	32.81±3.20	19.68±5.96	35.75±15.63	64.56±9.70	35.14±9.77	52.20±7.46	44.58±4.10	40.23±8.89	**75.14±0.21**
DFTJ	34.36±6.27	5.01±7.09	34.61±13.67	49.47±1.51	33.30±8.33	46.16±3.67	36.17±6.36	37.55±2.33	**75.80±1.92**
CSJ	43.28±4.99	38.35±2.65	34.42±26.37	49.23±1.29	39.85±7.90	49.13±8.09	42.84±10.67	43.00±4.42	**69.97±11.57**
NPJ	19.85±4.03	0.00±0.00	25.46±20.05	21.83±0.69	48.74±6.56	55.40±7.66	47.78±7.31	55.60±5.80	**69.82±0.46**
NCJ	32.03±4.54	0.00±0.00	14.46±12.47	40.43±2.96	43.98±8.61	52.83±6.54	49.24±7.89	50.96±8.28	**69.52±4.69**
AMJ	68.06±0.44	39.80±1.56	52.49±3.21	61.36±1.32	59.78±2.35	67.43±0.65	65.75±4.65	62.66±7.49	**82.64±3.64**
FMJ	28.83±5.50	0.00±0.00	18.59±26.29	23.74±0.20	54.53±9.50	59.42±3.46	58.29±2.95	57.30±12.08	**72.83±1.43**
Average	42.76±4.23	23.44±2.58	37.93±16.57	49.61±3.05	50.28±6.78	58.87±4.71	53.77±5.63	54.09±6.96	**75.52±2.97**

TABLE VI Few-Shot Recognition Accuracy (%) of Different Methods at Different Labeled Ratios

Ratio	Accuracy (%)								
	ResNet	AlexNet	VGG	DenseNet	SimCLR	MoCo v2	BYOL	VICReg	Ours
0.01	40.06±3.25	31.99±0.42	42.32±12.50	45.31±1.64	50.16±3.80	58.42±1.14	53.14±1.71	53.33±1.06	**75.13±1.59**
0.02	54.80±0.84	60.53±1.83	52.76±20.91	56.37±0.22	60.61±1.07	65.72±0.35	60.91±0.78	63.46±1.56	**80.52±0.28**
0.03	59.63±0.95	69.20±1.71	69.72±1.20	64.14±1.18	61.94±0.11	66.63±0.98	61.76±1.58	65.82±1.37	**82.64±0.13**
0.04	65.84±1.28	69.55±1.27	41.90±20.90	50.26±24.79	63.70±1.73	66.86±0.74	63.34±0.91	66.57±0.85	**80.71±0.32**
0.05	65.22±0.85	72.35±0.54	75.26±0.14	65.51±2.33	65.57±0.81	69.06±0.35	64.39±0.49	68.51±0.85	**83.20±0.29**
0.06	63.60±0.51	73.70±0.21	74.55±0.66	66.32±0.77	66.77±0.50	69.30±0.34	65.88±0.26	68.47±0.54	**83.86±0.52**
0.07	67.42±0.68	74.95±0.48	75.53±0.27	68.20±0.37	67.15±0.51	69.09±1.05	66.00±0.43	69.06±0.47	**84.48±0.50**
0.08	69.36±0.23	72.59±0.95	62.11±19.26	62.04±11.97	67.86±0.61	69.87±0.96	65.48±0.63	69.81±0.48	**83.83±0.85**
0.09	68.38±0.69	74.31±0.76	77.89±0.43	68.52±0.61	68.09±0.27	69.83±0.62	66.16±0.50	69.23±0.67	**84.78±0.32**
0.1	70.03±0.74	74.13±0.39	76.72±1.71	68.45±1.57	68.60±0.84	69.99±0.43	66.23±0.28	70.45±1.20	**84.76±0.59**

TABLE VII Ablation Study of Modality Contribution

Variant	OA(%)	Precision(%)	Recall(%)	F1(%)	kappa(%)
Full model	75.13±1.59	78.83±1.24	75.13±1.59	75.52±1.53	72.02±1.79
w/o Pretrain	58.25±2.26	65.48±2.71	58.25±2.26	58.68±2.58	53.03±2.54
w/o Signal branch	44.67±3.75	52.84±4.17	44.67±3.75	45.74±4.24	37.75±4.21
w/o Image branch	60.89±1.89	65.26±2.15	60.89±1.89	60.90±1.07	56.00±2.12

References

[1] S. Haykin, “Cognitive radar: a way of the future,” IEEE Signal Processing Magazine, vol. 23, no. 1, pp. 30–40, 2006, doi: 10.1109/MSP.2006.1593335.

[2] S. Z. Gurbuz, H. D. Griffiths, A. Charlish, M. Rangaswamy, M. S. Greco, and K. Bell, “An Overview of Cognitive Radar: Past, Present, and Future,” IEEE Aerospace and Electronic Systems Magazine, pp. 6–18, Dec. 2019, doi: 10.1109/MAES.2019.2953762.

[3] H. Zhou, Z. Wang, R. Wu, X. Xu, and Z. Guo, “Jamming recognition algorithm based on variational mode decomposition,” IEEE Sensors Journal, vol. 23, no. 15, pp. 17341–17349, 2023, doi: 10.1109/JSEN.2023.3283397.

[4] R. Zhang, C. Li, J. Zhang, and S. Lyu, “Radar jamming signal recognition algorithm based on multi-feature fusion,” Digital Signal Processing, vol. 158, art. 104950, 2025, doi: 10.1016/j.dsp.2024.104950.

[5] Y. Li, M. Dong, and R. Kothari, “Classifiability-based omnivariate decision trees,” IEEE Transactions on Neural Networks, vol. 16, no. 6, pp. 1547–1560, 2005.

[6] Q. Zhao and J. C. Principe, “Support vector machines for SAR automatic target recognition,” IEEE Transactions on Aerospace and Electronic Systems, vol. 37, no. 2, pp. 643–654, 2001.

[7] S. Paul, D. P. Mukherjee, P. Das, A. Gangopadhyay, A. R. Chintha, and S. Kundu, “Improved random forest for classification,” IEEE Transactions on Image Processing, vol. 27, no. 8, pp. 4012–4024, 2018.

[8] T. Kuang, H. Chen, L. Han, R. He, W. Wang, and G. Ding, “Abnormal signal recognition with time-frequency spectrogram: A deep learning approach,” arXiv preprint arXiv:2205.15001, 2022.

[9] S. Chen, X. Cui, M. Li, C. Tao, and H. Li, “SAR image active jamming type recognition method based on deep CNN model,” Journal of Radars, 2022.

[10] B. Lang and J. Gong, “JR-TFViT: A lightweight efficient radar jamming recognition network based on global representation of the time–frequency domain,” Electronics, vol. 11, no. 17, art. 2794, 2022.

[11] Q. Lv, Y. Quan, W. Feng, M. Sha, S. Dong, and M. Xing, “Radar deception jamming recognition based on weighted ensemble CNN with transfer learning,” IEEE Transactions on Geoscience and Remote Sensing, vol. 60, art. 5607516, 2022.

[12] P. Li, J. Yang, and J. Lin, “Radar compound jamming recognition based on image segmentation and fused attention residual network,” Sensors, vol. 25, no. 7, art. 2124, 2025.

[13] X. Chen, Y. Liu, and C. Wang, “Multi-domain fusion network for active jamming recognition in cognitive radar,” Remote Sensing, vol. 17, no. 10, art. 1723, 2025, doi: 10.3390/rs17101723.

[14] Z. Hou, K. Wang, J. Zhang, H. Wang, and K. Lu, “Jamming recognition of carrier-free UWB cognitive radar based on MA-Net,” IEEE Transactions on Instrumentation and Measurement, vol. 72, art. 8504413, 2023.

[15] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, “Multimodal machine learning: A survey and taxonomy,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 41, no. 2, pp. 423–443, 2019, doi: 10.1109/TPAMI.2018.2798607.

[16] Y.-X. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, “Generalizing from a few examples: A survey on few-shot learning,” ACM Computing Surveys, vol. 53, no. 3, pp. 1–34, 2020.

[17] J. Du, W. Fan, C. Gong, J. Liu, and F. Zhou, “Aggregated-attention deformable convolutional network for few-shot SAR jamming recognition,” Pattern Recognition, vol. 146, art. 109990, 2024.

[18] R. Luo, Z. Liu, Y. Wang, H. Li, and P. Zhang, “Few-shot radar jamming recognition network via complete information mining,” IEEE Transactions on Aerospace and Electronic Systems, vol. 60, no. 5, pp. 5157–5172, 2024.

[19] X. Ding, Y. Zhang, G. Li, X. Gao, N. Ye, D. Niyato, and K. Yang, “Few-shot recognition and classification framework for jamming signal: A CGAN-based fusion CNN approach,” arXiv preprint arXiv:2311.05273, 2023.

[20] Z. Huang, S. Denman, A. Pemasiri, C. Fookes, and T. Martin, “Few-shot radar signal recognition through self-supervised learning and radio frequency domain adaptation,” in Proc. IEEE Int. Workshop on Machine Learning for Signal Processing (MLSP), 2025.

[21] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in Proc. 37th Int. Conf. Machine Learning (ICML), PMLR, vol. 119, pp. 1597–1607, 2020.

[22] X. Chen, H. Fan, R. Girshick, and K. He, “Improved baselines with momentum contrastive learning,” arXiv preprint arXiv:2003.04297, 2020.

[23] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in Proc. Int. Conf. Learning Representations (ICLR), 2017.

[24] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), pp. 16000–16009, 2022.

[25] Z. Hou, X. Liu, Y. Cen, Y. Dong, H. Yang, C. Wang, and J. Tang, “GraphMAE: Self-supervised masked graph autoencoders,” arXiv preprint arXiv:2205.10803, 2022.

[26] J.-B. Grill et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in Advances in Neural Information Processing Systems, vol. 33 (NeurIPS 2020), 2020.

[27] A. Bardes, J. Ponce, and Y. LeCun, “VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning,” in International Conference on Learning Representations (ICLR), 2022.